Candidate Instructions for AI Native Interview

1. Goal of the exercise

We want to understand how you use AI-assisted coding tools to systematically solve a problem end-to-end: clarifying requirements, designing a solution, implementing it, and adding tests. This is specifically about your **AI-native workflow**, not about typing speed.

2. Logistics and environment

- Duration: We will spend about **45 minutes** on this exercise.
- Environment:
  - Please use your **own laptop** with your usual development setup.
  - You are encouraged to use an AI-assisted IDE or coding environment (e.g., Cursor, Antropic's antigravity, Copilot, etc.) with sufficient AI credits for the session.
- Screen sharing:
  - You will **share your screen** for the entire exercise so we can see:
    - How you turn the plain requirements into a more detailed spec.
    - How you prompt and interact with your AI tools.
    - How you navigate between design, coding, and testing.

3. What you will build

- At the start of the session, we will give you **one problem** in the form of the plain requirements.
- The scope is intentionally small so that you can:
  - Clarify assumptions.
  - Produce a minimal but coherent design.
  - Implement core functionality.
  - Add meaningful test coverage.

4. How we'd like you to work (spec-driven, not "vibe coding")

During the session, please follow this general flow:

1. **Clarify and restate requirements (5–10 minutes)**

   Read the initial requirements.

   Ask any clarifying questions you have.

   In your editor (e.g., a `README.md` or `DESIGN.md`), write a brief **structured spec**:

      Problem restatement in your own words.

      Key use cases and non-goals for the 45-minute version.

      Proposed API or interfaces (request/response shapes, main functions).

      Initial test scenarios you plan to cover.

2. **High-level design (5–10 minutes, can overlap with coding)**

   - Sketch a simple design in the same file:

      Main components/classes/modules.

      Data models and key fields.

      How you will structure tests.

   - You are welcome to ask your AI IDE to help you draft this, but:

      Please **review and edit** the AI output.

      Talk through what you're accepting/rejecting and why.

3. **Implementation and tests (20–30 minutes)**

   - Implement the minimal viable version of the solution that fulfills your spec.
   - Add automated tests (e.g., unit tests) that align with the test scenarios you wrote earlier.
   - Use AI to help with boilerplate or suggestions, but:
     - Keep control of the architecture and interfaces.
     - Explain your reasoning as you go:
       - ♣ Why you chose a particular data model.
       - ♣ How your agent logic or rules evaluation works.
       - ♣ How your tests validate behavior and edge cases.

4. **Quick walkthrough (last 3–5 minutes)**

- Run the tests and/or a small demo.
- Walk us through:

  The spec you wrote.

  The main code structure.

  How you used AI tools during the exercise.

  What you would do next with more time (e.g., refactors, better error handling, more realistic integrations).

5. Use of AI tools

- You are explicitly **encouraged** to:
  - Ask your AI assistant to help refine the spec, propose designs, or generate code.
  - Iterate on prompts when the first response isn't what you want.
  - Use AI to generate tests, then review and adjust them.
- We will be paying attention to:
  - How you **frame prompts** (context, constraints, examples).
  - How you **critically evaluate** AI output.
  - How you keep the implementation aligned with your written spec.

6. Deliverables after the interview

At the end of the interview (or shortly after), please:

1. **Package the project artifacts**:
   - Source code.
   - `README.md` with:
     - Problem restatement.
     - Design summary.
     - How to run the service/tests.
     - Any assumptions or shortcuts taken for the exercise.
   - Any additional docs or notes you created (`DESIGN.md`, etc.).
2. **Send via email**:
   - Either:

        - A link to a public or private Git repo (GitHub, GitLab, etc.), or
        - A compressed archive (e.g., `.zip`) attached to the email.
    - Please remove any personal API keys, secrets, or credentials before sharing.